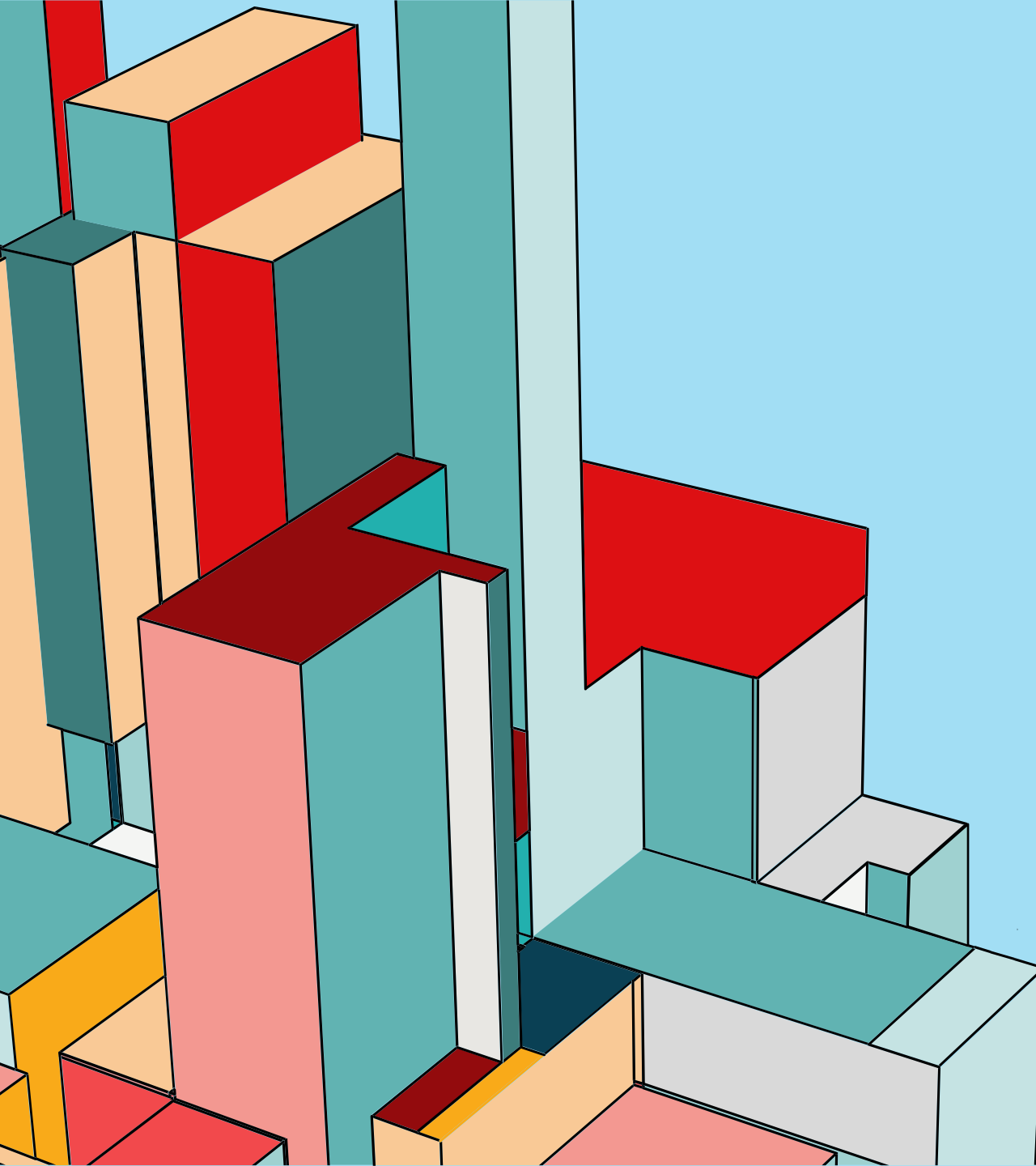


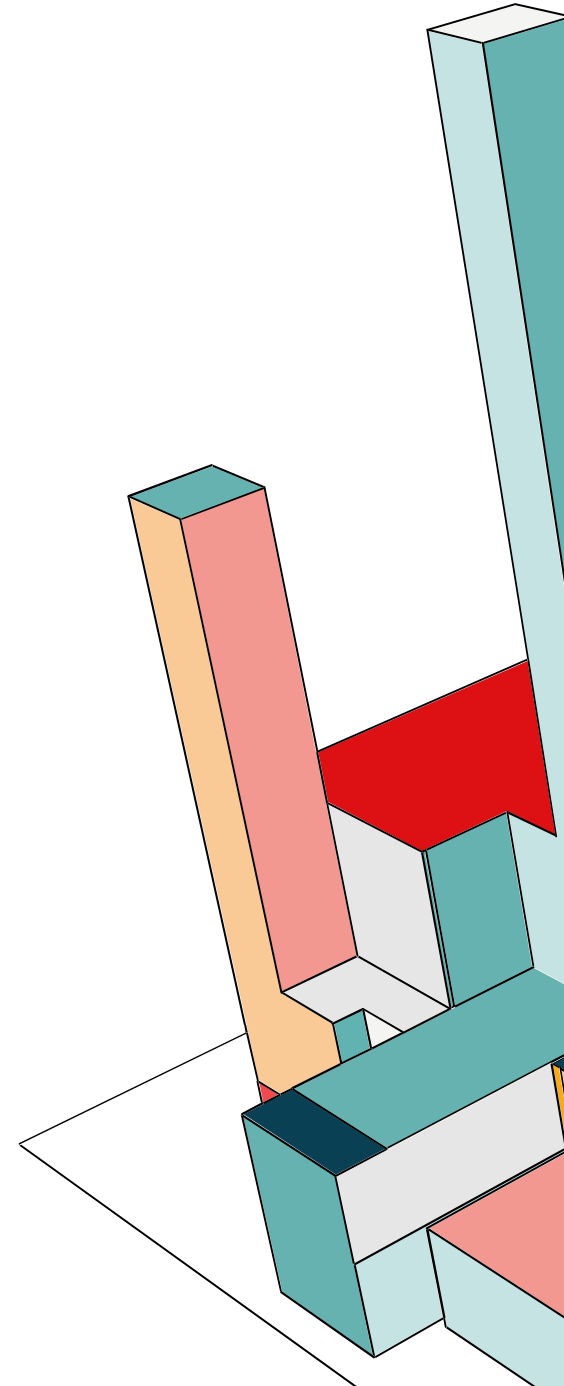


LECTURE 12 **GREEDY ALGORITHMS AS APPROXIMATIONS**

CSIS 2300 Introduction to
Algorithms, Spring 2026
Dr. Dan DiTursi

AGENDA

- Quick review from last class
- NP-complete problems (briefly)
- Vertex Cover
- Traveling Salesman Problem



RECAP OF LAST CLASS

Bellman-Ford algorithm

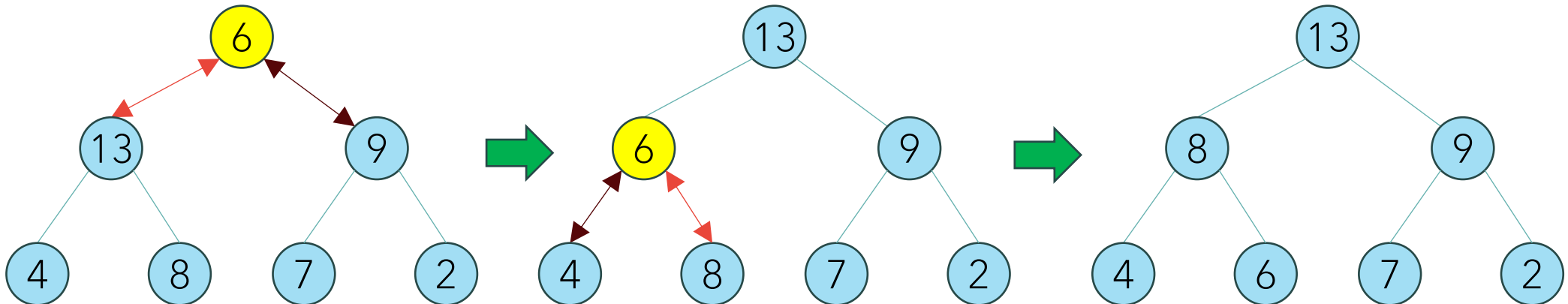
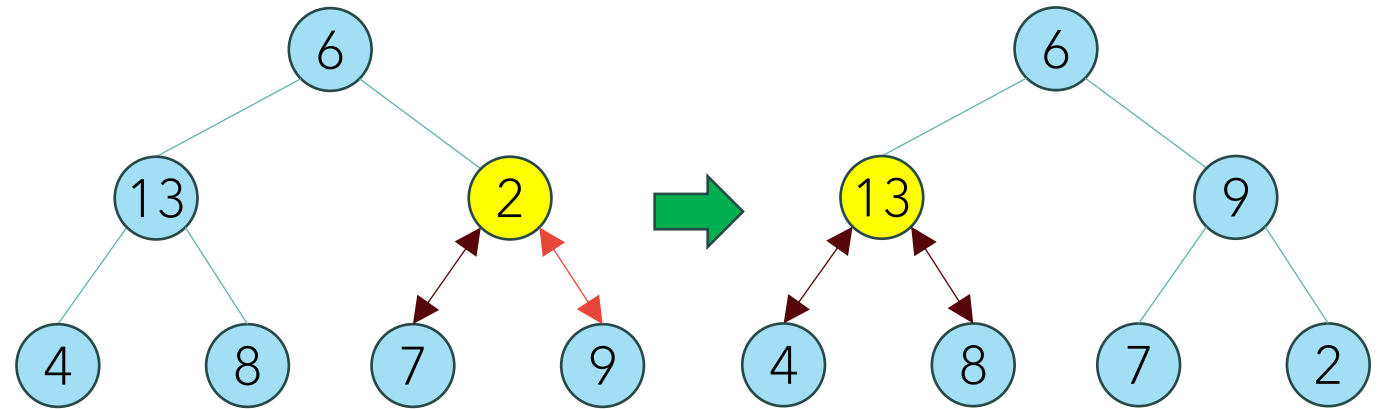
- Single-source shortest path when there are negative weights
- Detects negative cycles as well
- $O(VE)$ runtime (notably slower than Dijkstra)

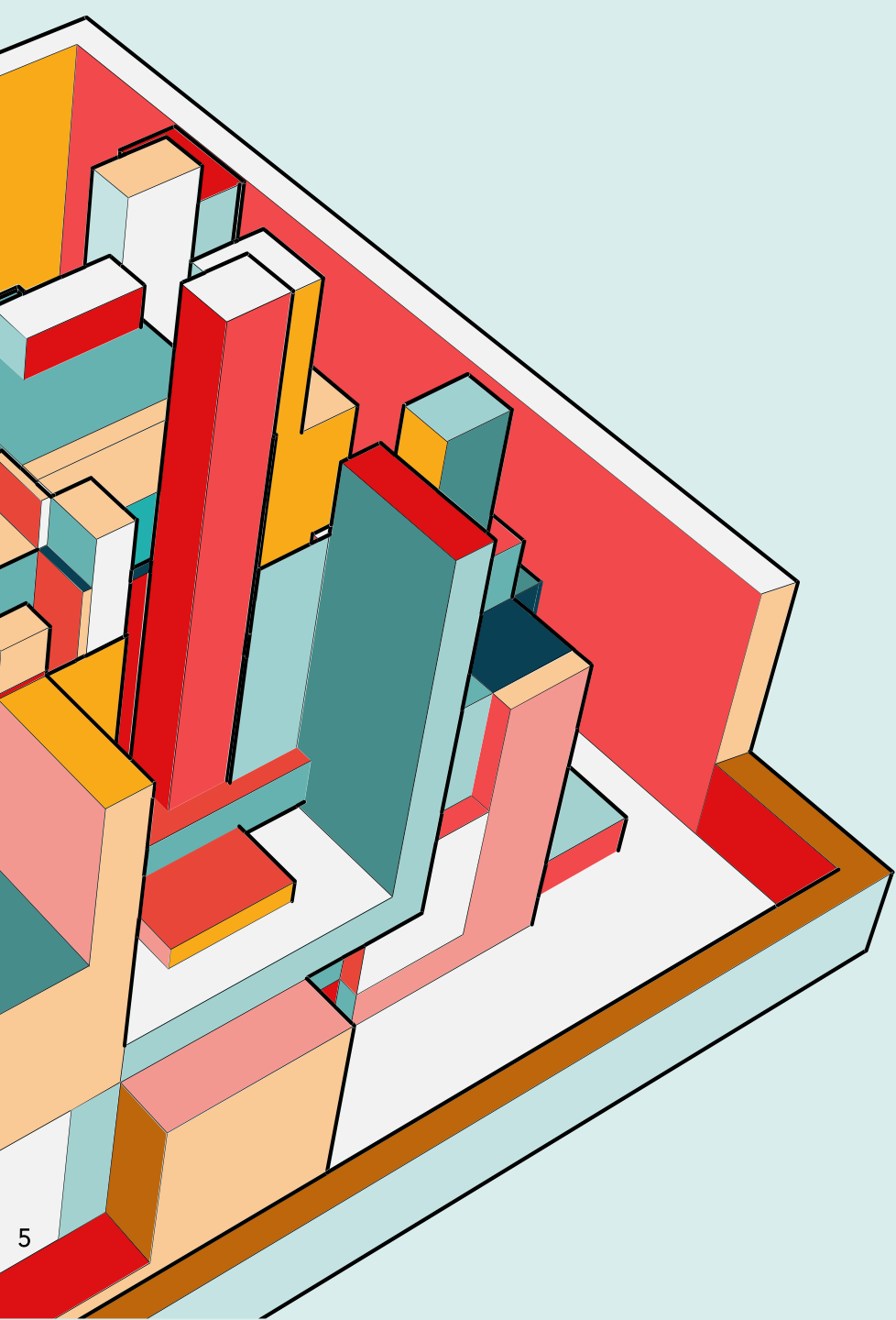
Heaps & heapsort

- Tree-based data structure where the root of every subtree is greater than all descendants (and tree is essentially complete)
- Used for priority queues. $O(1)$ query of highest value; $O(\log n)$ removal of highest value; $O(\log n)$ insertion of new value
- Creation of heap from scratch in $O(n)$ time using bottom-up approach

BOTTOM-UP HEAP CONSTRUCTION

The "heapify down" operation is applied to every non-leaf node in reverse order.





“HARD” PROBLEMS

HARD PROBLEMS VS. LOUSY SOLUTIONS

You are presented with a new-to-you problem and a solution for that problem that runs `v e r y s l o w l y`.

There are a few possibilities:

- Poor implementation. This is outside the scope of this course – go bug your PSoft professor.
- Poor algorithm – there are better solutions available that you just haven't found yet.
- Hard problem – there is literally no way to solve a problem of the given size quickly.

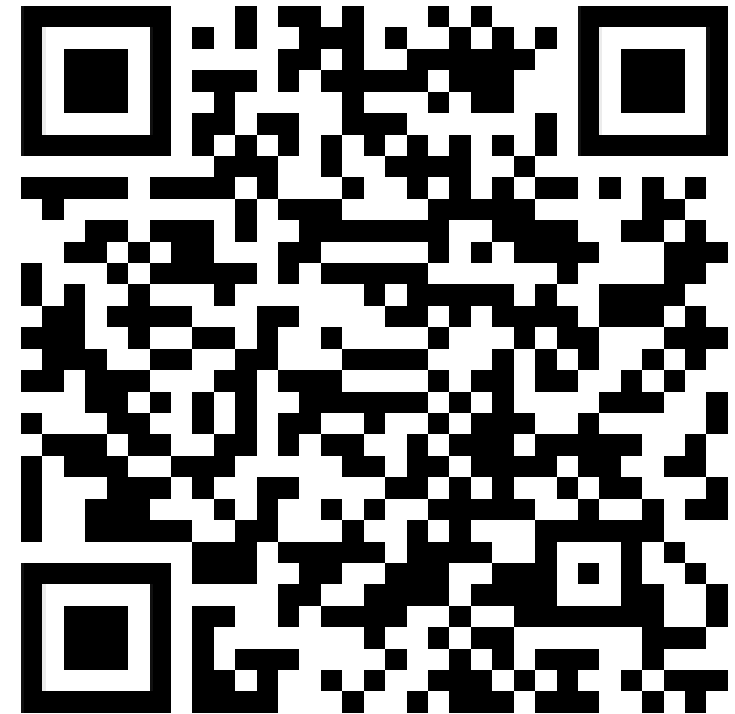
One of the reasons you're taking this course is to develop skill at distinguishing between the latter two cases.

AN EXAMPLE PROBLEM

You are given an ordered list of arbitrary signed integers. You are asked to find the subsequence of integers in the list with the highest sum.

You are given a program that solves the problem. On a list of 100 integers, it finishes in 6 seconds. On a list of 1000 integers, it takes over an hour and a half. Looks like it's an $O(n^3)$ algorithm. Surely there is better?

What is the runtime of the best possible algorithm for this problem? Answer in the Submittity poll now.



AN EXAMPLE PROBLEM

You are given an ordered list of arbitrary signed integers. You are asked to find the subsequence of integers in the list with the highest sum.

Hopefully, you recall this problem from Exam 1; a divide-and-conquer algorithm solves this in $O(n \log n)$ time.

However, even that is not optimal! There is a solution using dynamic programming that runs in $O(n)$ time!

The point here is that without some sort of proof, we don't know that a problem is hard – all we know is that we don't have a better solution for it.

WHAT DOES SUCH A PROOF LOOK LIKE?

Think back to FOCS for a moment. How do you show, for a given proposition P , that $\neg \exists x P(x)$?

 That's actually really hard to do!

There are cases where we can prove some limits – for example, comparison-based sorting, where we can prove an $O(n \log n)$ lower bound on the number of comparisons.

One useful idea is called a *proof by reduction* – you show that problem A is “just as hard” as problem B by demonstrating that you can transform any instance of B into an instance of A, so that a general algorithm for A also provides an algorithm for B.

Throwback to lecture 2...

- The common classes of functions we work with are:
 - $O(1)$ - constant (*array access*)
 - $O(\log n)$ - logarithmic (*binary search in a sorted array*)
 - $O(n)$ - linear (*search in an unsorted array*)
 - $O(n \log n)$ - loglinear (*good sorting algorithms like mergesort*)
 - $O(n^2)$ - quadratic (*poor sorting algorithms like bubble sort*)
 - $O(n^c)$ - higher-order polynomials (*some parsing algorithms*)
 - $O(n^{\log n})$ - quasipolynomial (*old primality test*)
 - $O(2^n)$ - exponential (*Traveling Salesman Problem*)
 - $O(n!)$, $O(n^n)$, $O(n^{2^n})$, and even worse - Don't go there.

CATEGORIES OF DIFFICULTY

Computer scientists have defined a number of different “complexity classes”, but far and away the most important of these are P and NP .

P = the set of all problems in which a solution to any instance can be generated in polynomial time or better

NP = the set of all problems in which a solution to any instance can be verified in polynomial time

The single biggest open question in theoretical computer science is simply this: Is $P \subset NP$ or is $P = NP$?

Put another way: Are there any problems for which it is possible to verify solutions quickly but impossible to generate solutions quickly?

THE COOK-LEVIN THEOREM

It may not be immediately obvious why $P = NP$ is such a big deal. It comes back to proofs by reduction.

In 1971, Stephen Cook published a paper proving that every NP problem was reducible to one particular problem (the *Boolean satisfiability problem*, or SAT). Leonid Levin was doing similar, independent work in Russia at the same time.

SAT is the first example of an *NP-complete* problem. People quickly started proving that many other common problems in computer science are also NP-complete.

NP-COMPLETENESS AND “DIFFICULTY”

NP-complete problems are considered “hard” problems, because we don’t know of any polynomial-time algorithms that generate solutions to these problems.

OBJECTION! The entire first part of this lecture was about the idea that “we don’t know of an algorithm” and “no algorithm is possible” are two very different things!

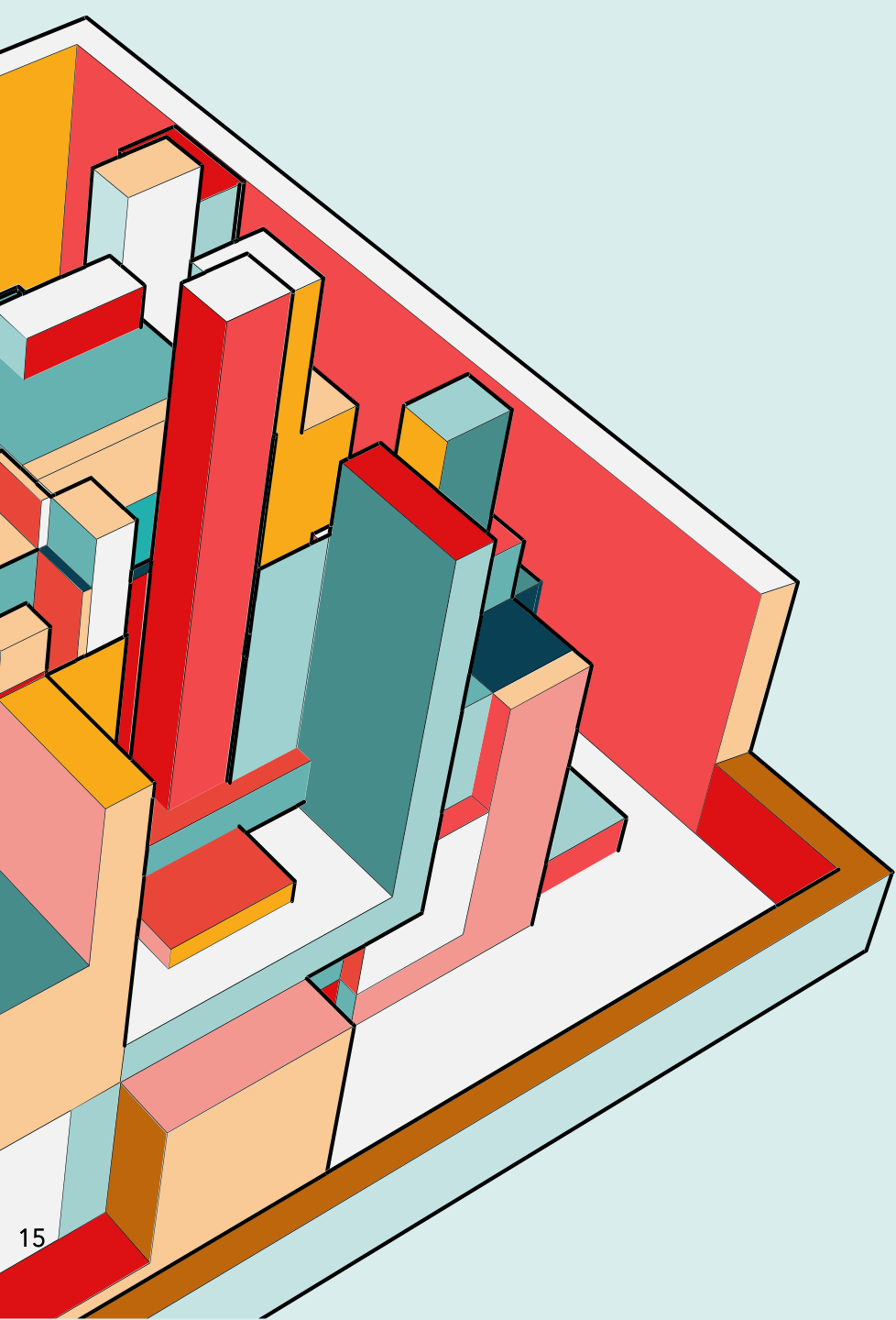
In this case, many computer scientists have been trying for decades to prove that one – just one! – of these NP-complete problems doesn’t have a polynomial-time algorithm. No luck.

Simultaneously, many computer scientists have been trying for decades to find a polynomial-time algorithm to solve one – just one! – NP-complete problem. Also no luck.

NP-COMPLETENESS AND “DIFFICULTY”

What we're left with is a pseudo-guarantee: If you run into an NP-complete problem, you can be assured that no one knows of a fast general solution.

The industry consensus is that $P \neq NP$ – last time I looked at a survey, about 80% of experts believed that, versus 15% that thought $P = NP$ and 5% who think it will be like the Parallel Postulate: Neither conclusion is forced by the axioms of mathematics, and assuming either conclusion can lead you somewhere interesting.



VERTEX COVER

(ONE VARIANT OF) FACILITY PLACEMENT

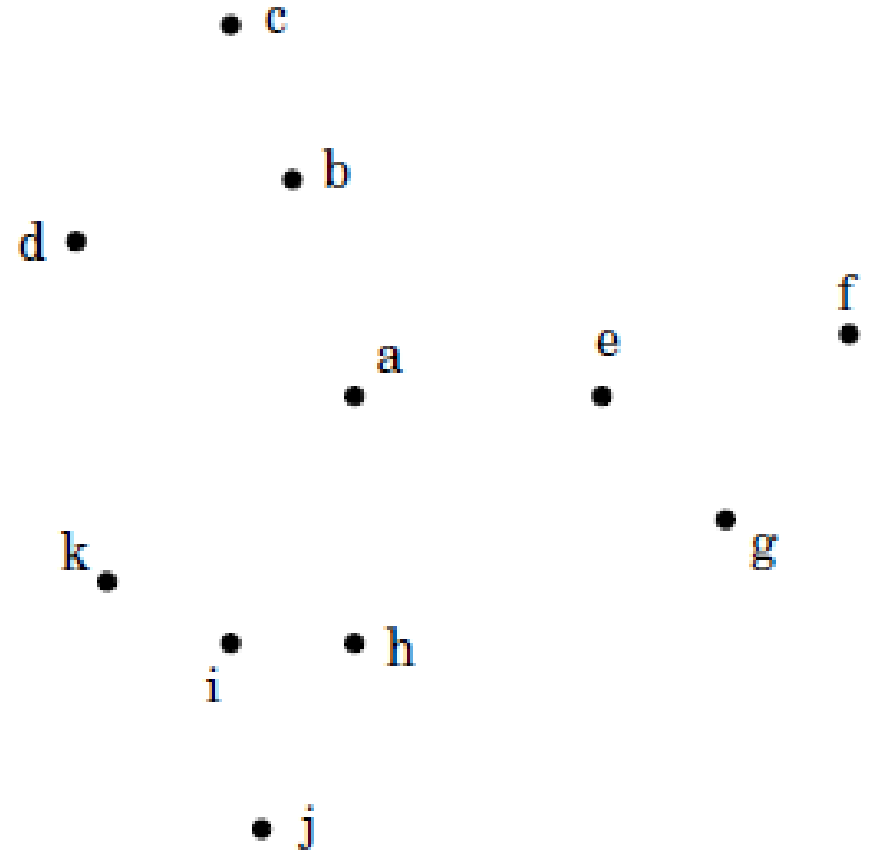
Consider a set of towns laid out on a plane.

A company wants to move into this area.

They need distribution centers. Their restrictions are (1) each distributor must be placed in a town and (2) all towns must be no more than d distance from a distributor.

Subject to these conditions, they would like to set up as few centers as possible.

How should they tackle the problem?

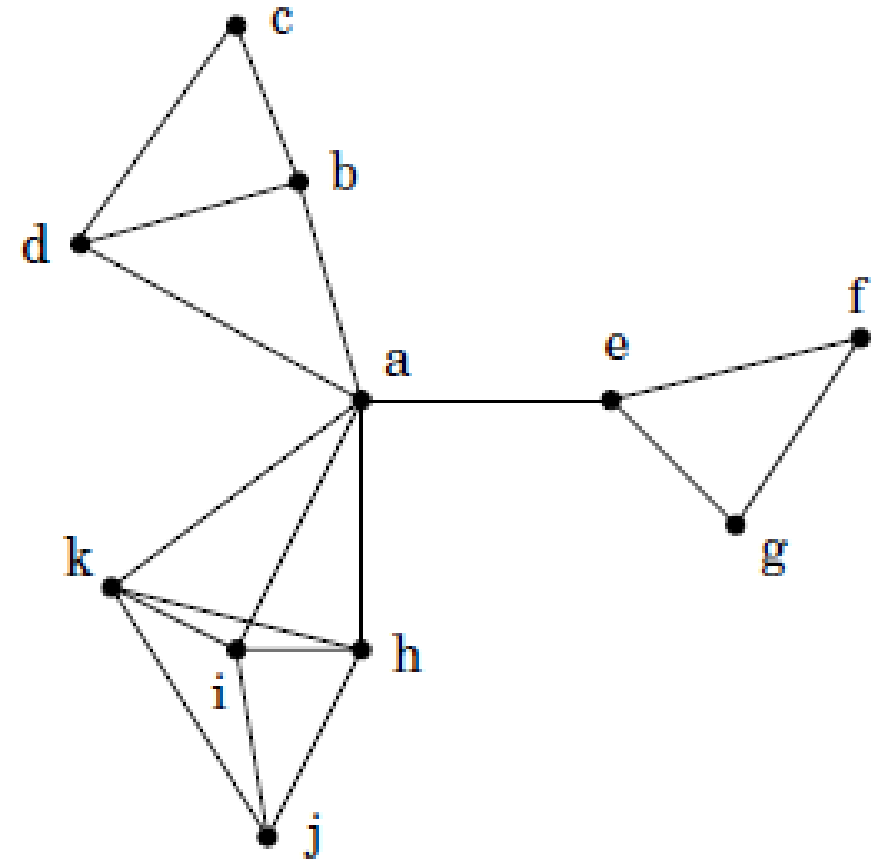


(ONE VARIANT OF) FACILITY PLACEMENT

An obvious first step is to generate a graph from the points, connecting those vertices that “cover” each other.

Now, given such a graph, what algorithm might we use to decide where to place the distribution centers?

One obvious approach would be a greedy algorithm: Repeatedly select the towns that cover the maximum number of uncovered towns, until all towns are covered.

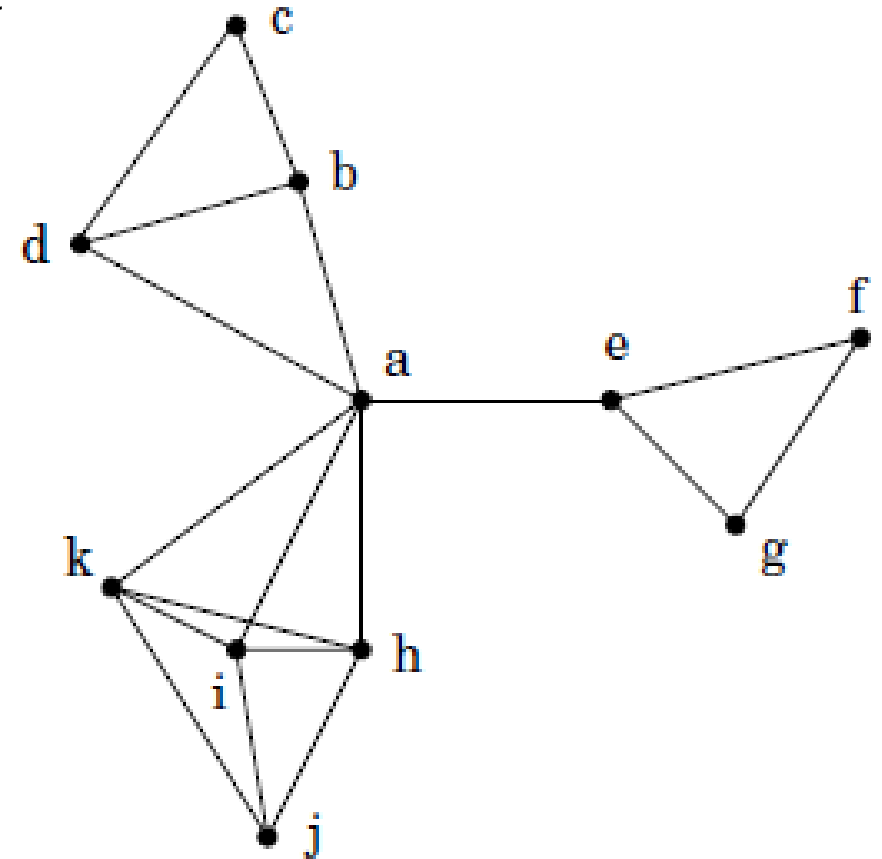


(ONE VARIANT OF) FACILITY PLACEMENT

A greedy algorithm will select node *a* first. It will then select either *f* or *g*, and then it must take *c* and *j*, for four total centers.

However, it should be clear from inspection that we can produce a solution that uses only three distribution centers: *b, e, h* is one such solution.

Thus, this greedy algorithm is incorrect – it does not always return an optimal solution.



STATING THE PROBLEM MORE GENERALLY

Given an undirected graph G , let $\mathcal{N}(v)$ indicate the *neighborhood* of vertex v : the set of nodes connected to v by an edge. For convenience here, v is included in its own neighborhood.

We seek a minimum-size subset of vertices $S \subseteq V$ such that

$$V = \bigcup_{s \in S} \mathcal{N}(s)$$

In other words, we want every node in the graph to either be in S or be connected to a node in S .

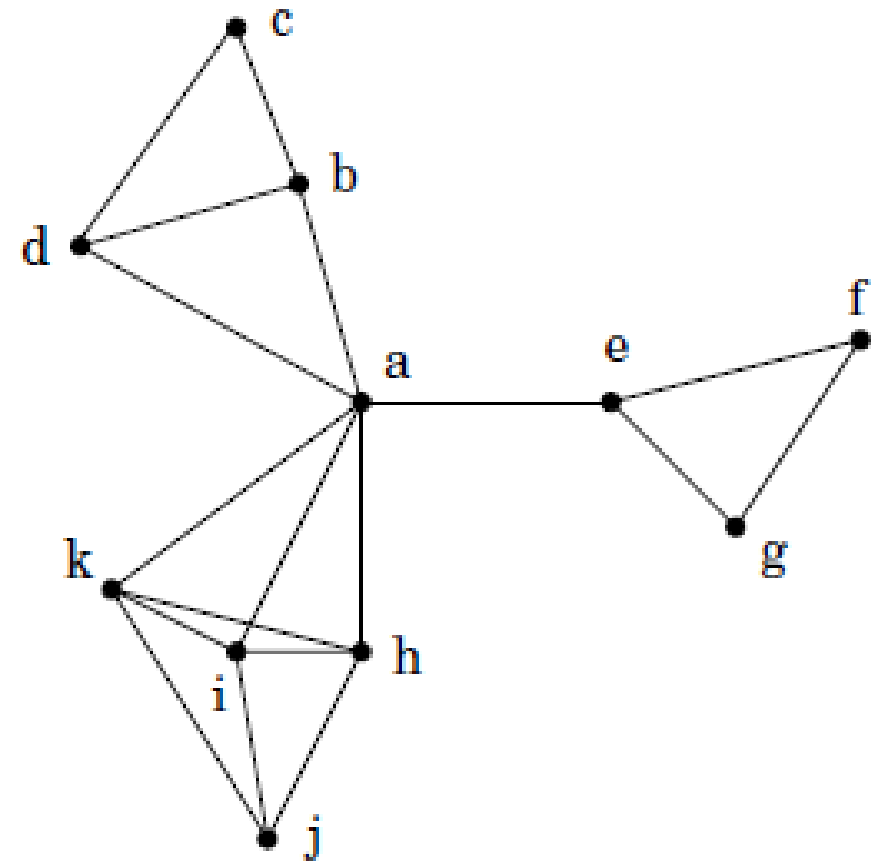
This is called the *Vertex Cover* problem. And it is NP-complete.

THE NP-COMPLETE WALL

Since Vertex Cover is NP-complete, we “know” that there is no polynomial-time algorithm that will give an optimal answer.

Let’s once again have a look at the greedy solution. Recall that it produced an answer with four distribution centers on this graph, whereas the optimal answer used three.

That’s really not too bad – it is only 1.33x the size of the optimal solution. Given how much faster the greedy algorithm will be, we’d probably be willing to accept that... if we could somehow guarantee we’d always be that close!



APPROXIMATION GUARANTEES

An *approximation guarantee* is a mathematical proof that a particular non-optimal algorithm always produces a result that is close to an optimal answer.

The ideal case for these is a *constant factor* guarantee, e.g. “we will never be worse than twice the size of the best answer”

For our greedy Vertex Cover example, the best we can do is a *logarithmic factor* guarantee:

Claim: Given an instance of Vertex Cover with n vertices, if an optimal solution requires k vertices, then the greedy solution requires no more than $k \ln n$ vertices.

APPROXIMATION GUARANTEE FOR VERTEX COVER

Claim: Given an instance of Vertex Cover with n vertices, if an optimal solution requires k vertices, then the greedy solution requires no more than $k \ln n$ vertices.

This is a bit of an extraordinary claim! We don't know what k actually is, nor do we have a way of easily finding it. Yet we can still claim that we'll be within a factor of $\ln n$ of it.

And it's all thanks to the Pigeonhole Principle!

PROOF OF APPROXIMATION GUARANTEE

Proof: Let n_t be the number of vertices still uncovered after the greedy approach has selected t vertices to be part of S .

The optimal solution of k vertices must cover all of these n_t vertices; therefore, by Pigeonhole, at least one of the optimal vertices will cover n_t/k of them. The greedy algorithm will thus also make a selection that covers at least that many nodes.

This gives us $n_{t+1} \leq n_t - n_t/k = n_t(1 - 1/k)$.

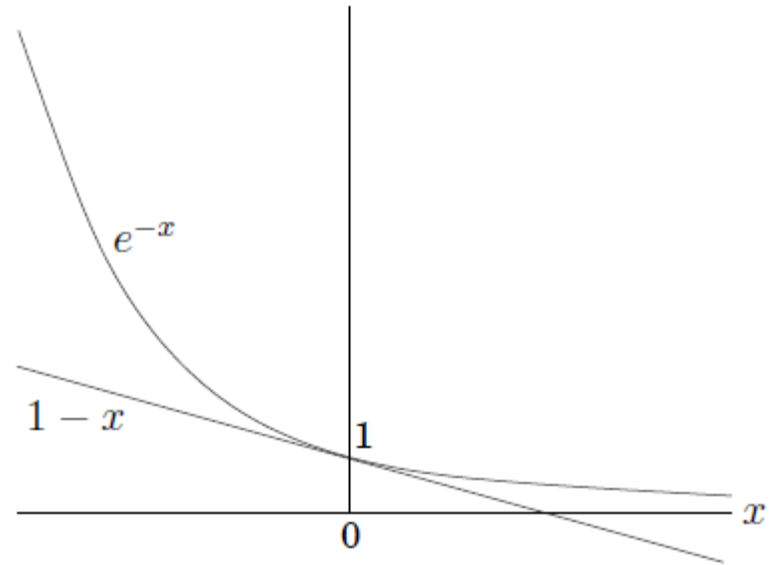
We can chain this idea backwards from n_t all the way to n_0 :

$$n_t \leq n_0(1 - 1/k)^t$$

PROOF CONT'D

$n_t \leq n_0(1 - 1/k)^t$; Note that $n_0 = n$.

There is a nice identity we can use here: $\forall x \neq 0, 1 - x < e^{-x}$



Substituting in: $n_t < n(e^{-1/k})^t = ne^{-t/k}$

Finally, consider what happens when $t = k \ln n$:

$$n_t < ne^{-(k \ln n)/k} = ne^{-\ln n} = n \cdot \frac{1}{n} = 1$$

Since n_t is strictly less than 1, it must be that $n_t = 0$ - there are no elements remaining to be covered after the greedy algorithm has made $t = k \ln n$ selections. ■

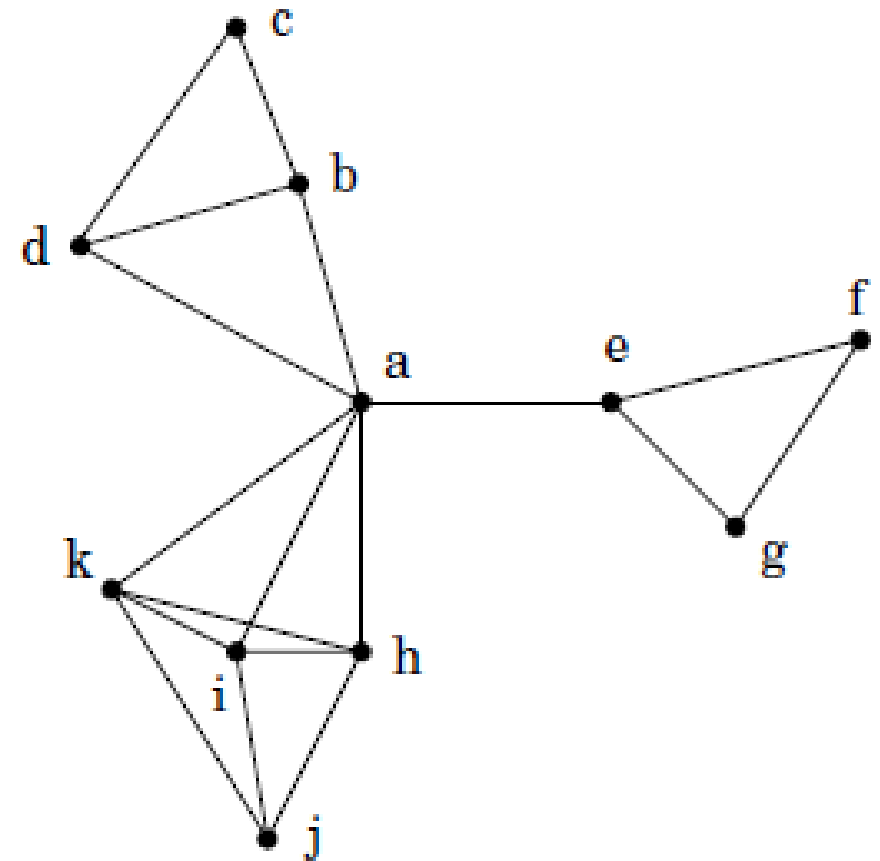
TIGHTNESS OF GUARANTEE

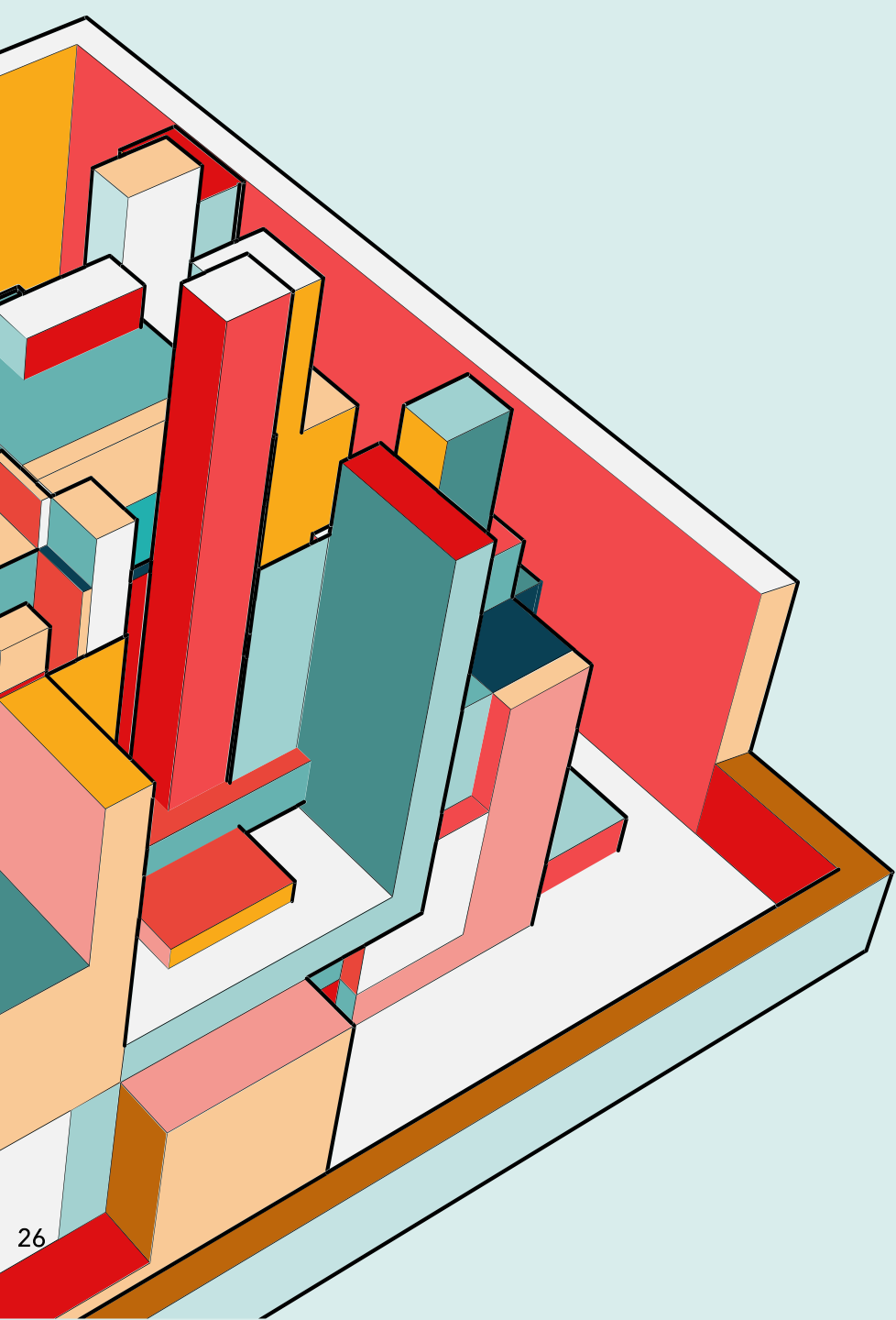
Recall that in our example, the greedy solution was 1.33x worse than the optimal.

$n = 11$ here, and $\ln 11 \approx 2.4$, so in this case we significantly outperform the guarantee.

One key question when producing these guarantees is the *tightness* of the bound: Are there instances of the problem where the difference from the optimal is at or close to the proven bound.

For this approach to VC, the bound is *tight*; there are instances where the optimal solution is $\ln n$ better than ours.





TRAVELING SALESMAN PROBLEM

THE TRAVELING SALESMAN PROBLEM

We have a weighted graph G .

Starting from any vertex v , we would like to visit every other vertex exactly one time and return to v .



Note that even determining the existence of such a path (called a *Hamiltonian circuit*) in a general graph is NP-complete.

But the TSP is (on its face) even harder, as it asks for us to find the shortest (i.e. minimum-weight) such cycle.

Since we “know” there won’t be a fast algorithm that provides an exact solution, we should look for an approximation.

HOW GOOD CAN THE APPROXIMATIONS BE?

One reason to look closely at TSP is that there's a nice, fairly simple proof that as long as $P \neq NP$, then there is no constant-factor polynomial-time approximation algorithm for it.

Proof: Assume, for contradiction, that there is such an approximation algorithm for TSP, with positive constant c .

By definition, this means that, on every instance of the problem, the approximation algorithm produces a solution with weight no larger than cw^* , where w^* is the weight of the optimal solution.

We will show that this TSP approximation gives an actual solution to the Hamiltonian Circuit problem.

HOW GOOD CAN THE APPROXIMATIONS BE?

Let G be an arbitrary graph in which all edges have weight 1. Observe that if G has a Hamiltonian circuit, then its total weight is n (since it uses n edges to go to each node once & go home).

Now, construct a new graph G' that is a copy of G except that all missing edges are added with weight $cn + 1$. Observe that G' has many Hamiltonian cycles, since it is a complete graph.

There are two categories of solution to TSP in G' :

- If we can solve it using only edges from G , then it has length n .
- If we need any of the new edges, then it has length $> cn$.

HOW GOOD CAN THE APPROXIMATIONS BE?

There are two categories of solution to TSP in G' :

- If we can solve it using only edges from G , then it has length n .
- If we need any of the new edges, then it has length $> cn$.

Note that if a solution of the top variety exists then (a) it is an optimal solution to TSP in G' and (b) it is also a solution to Hamiltonian Circuit in G .

Recall that our TSP approximation is guaranteed to produce a solution with distance $\leq cn$. Thus, if one of the top solutions exists at all, that's what it will find, because all solutions of the bottom type are too long.

HOW GOOD CAN THE APPROXIMATIONS BE?

So, we run our TSP approximation on G' . One of two things happens:

- It finds a solution of length n . Then G has a Hamiltonian Circuit.
- It finds a solution of length $> cn$. Then G' must not have a circuit of length n , and G has no Hamiltonian circuit.

In other words, our assumed constant-factor polynomial-time TSP approximation solves Hamiltonian Circuit, which is an NP-complete problem. This contradicts the assertion that $P \neq NP$. So if $P \neq NP$, there cannot be such an approximation algorithm for TSP. ■

TWO GREEDY APPROACHES TO TSP

The simplest greedy algorithm is just to pick a starting node and at every step follow the lowest-weight edge that goes to an unvisited city.

This can be pretty terrible – your last steps may end up VERY long.

A slightly better approach looks somewhat like Kruskal's algorithm and relies on the idea that a Hamiltonian circuit contains precisely two edges that touch each vertex.

- Sort the edges in ascending order by weight.
- Repeat n times: Find the next edge in sequence that will not complete a cycle smaller than n and that will be a third edge for one of its endpoints. Add that edge to the circuit.

GUARANTEES?

Even the improved greedy algorithm has no guarantee that it's any good... unless we restrict the problem.

If we are actually solving TSP on a map, where the vertices represent points in the plane and the edges distances, then the modified-Kruskal algorithm gives solutions that have an approximation factor of $\frac{1}{2} (\lceil \log_2 n \rceil + 1)$.

Such instances are called *Euclidean*, and the key properties are that the edge weights obey these conditions:

- Symmetry: $w(u, v) = w(v, u)$
- Triangle Inequality: $w(u, v) \leq w(u, x) + w(x, v)$

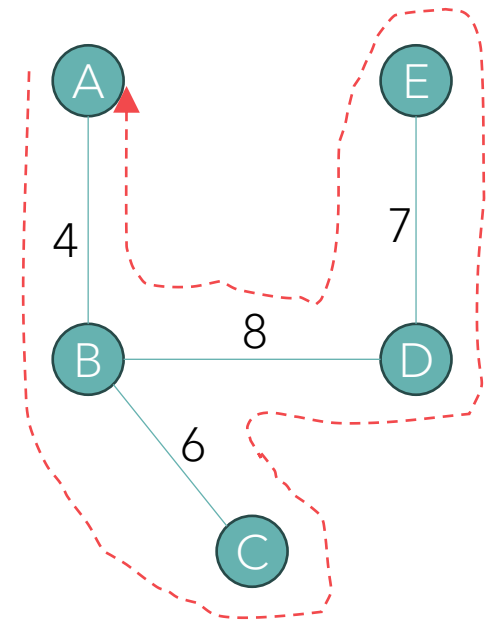
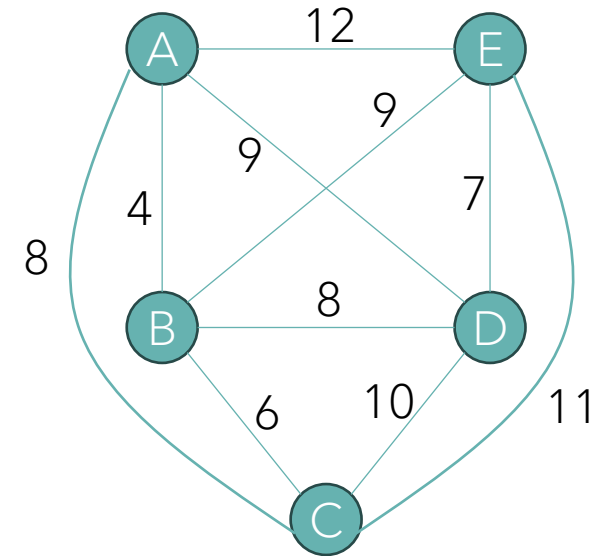
AN EVEN BETTER APPROACH

Instead of modifying Kruskal, why don't we just go ahead and make a minimum spanning tree?

Then we can just use the "maze algorithm" (i.e. depth-first search) to walk around the tree and get back to the beginning.

Observe that this uses every edge in the MST twice.

Also observe that this is not a valid solution to the problem, as it visits some vertices multiple times.



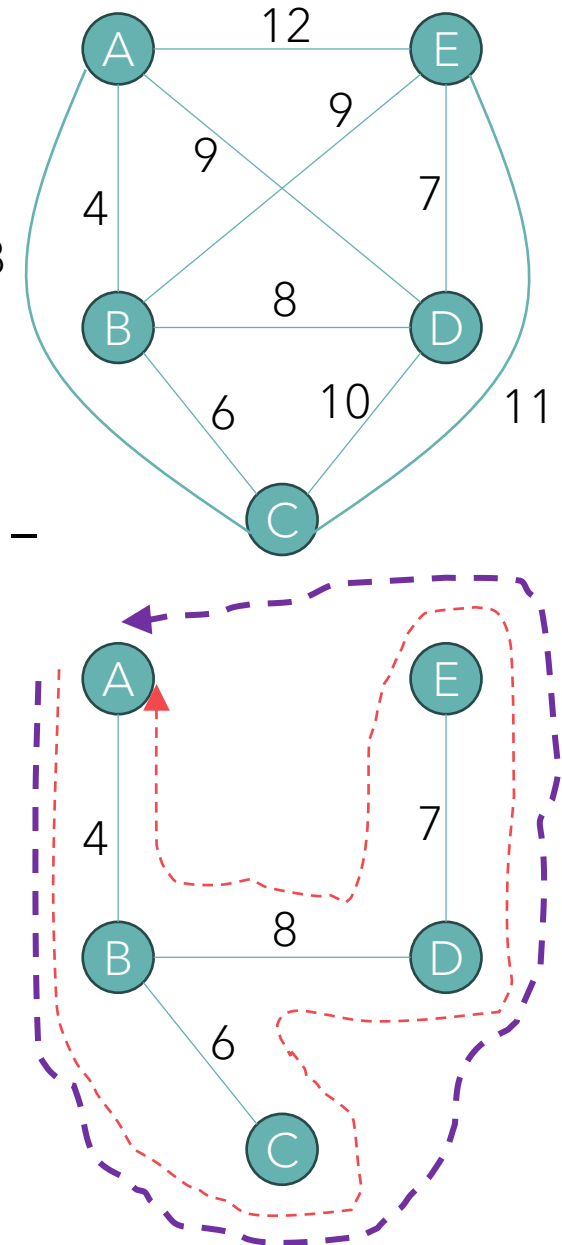
AN EVEN BETTER APPROACH

However, given this path, we can take “shortcuts”₈ by skipping vertices that we’ve already visited.

If the graph is Euclidean, we know (from the Triangle Inequality) that these really are shortcuts – they cannot make the path longer.

Since an optimal solution to TSP has to connect every vertex, we know its weight must be at least as much as the MST.

Our solution is no worse than double an MST. So “twice around the tree” is a 2-approximation to TSP... as long as the graph is Euclidean.



COMPLETE THE POLL!

Make sure to complete the left-hand poll – it's the second attendance poll for the day. On the right is the standard end-of-class feedback form. Have a good break!

